

OpenText™ eDirectory

NICI administration guide

Version : 25.4

PDF Generated on : June 22, 2026

Table of Contents

1. NICI administration guide	1
1.1. Overview	2
1.1.1. Understanding the NICI modules	3
1.2. What's new	5
1.2.1. Support for 8192 bit RSA encryption	6
1.2.2. FIPS mode for NICI	7
1.2.3. Operating NICI in FIPS mode	8
1.2.4. NICI SDI health check	9
1.2.5. Creating an inherited rights mask	10
1.2.6. Dependency change	11
1.3. Installing NICI	12
1.3.1. Checklist for installing NICI	13
1.3.2. Understanding the NICI installation	16
1.3.3. Prerequisites and considerations for installing NICI	17
1.3.4. System requirements	18
1.3.5. Installing NICI	19
1.3.6. Meeting the LSB compliant directory structure	22
1.3.7. Locating the NICI configuration files	23
1.3.8. Post-installation tasks	26
1.4. Upgrading NICI	34
1.4.1. Checklist for upgrading NICI	35
1.4.2. Prerequisites and considerations for upgrading NICI	38
1.4.3. Considerations for upgrading NICI on servers with multiple instances of OpenText eDirectory	39
1.4.4. Backing up and restoring the current configuration	40
1.5. Understanding the NICI keys	41
1.5.1. Types of keys	42

1.5.2. Managing the tree keys	45
1.5.3. Synchronizing the NCI SDI keys	47
1.5.4. Creating an AES 256-Bit tree key	48
1.5.5. NCI SDI health check	50
1.5.6. Merging trees	51
1.5.7. Diagnosing SDI key synchronization issues	52
1.6. Backing up and restoring NCI	53
1.6.1. Performing a backup	55
1.6.2. Restoring NCI	58
1.6.3. Automatic backing up and restoring NCI	62
1.7. Resolving errors	63
1.7.1. Error messages	64
1.8. Documentation updates	67
1.8.1. February 2018	68
1.8.2. January 2016	69
1.8.3. August 2008	70

1. NCI administration guide

The *Guide* describes the structure and functionality of Novell International Cryptographic Infrastructure (NCI), how to set it up and manage it. This guide also documents NCI error messages. This book is intended for network administrators. NCI is available with OpenText eDirectory, OpenText Identity Console, and Novell Client([Novell Client Documentation Website](#)).

1.1. Overview

Novell International Cryptography Infrastructure (NICI) is a cross-platform, policy-driven, independently certified, and extensible cryptography service. NICI is the cryptography module that provides keys, algorithms, various key storage and usage mechanisms, and a large-scale key management system.

NICI controls the introduction of algorithms and the generation and use of keys. NICI allows a single commodity version of security products to be produced for worldwide consumption that supports strong cryptography and multiple cryptographic technologies. Initial services built on this infrastructure are Directory Services (OpenText eDirectory), NetIQ Modular Authentication Service (NMAS), OpenText Certificate Server, and NetIQ SecretStore.

This document helps you to understand and administrator the NICI services which are used by various OpenText, or third-party products. A particular product might use NICI directly or indirectly via another module (`DLL` , `.so`).

NICI 3.2 (64-bit) is Federal Information Processing Standards (FIPS) 140-2 Inside validated to meet the security requirements of U.S. Federal agencies and customers with highly secure environments.



Caution

Many actions described in the document could potentially cause unrecoverable data loss and must be executed with the full knowledge of such an action. Most NICI problems, as well as solutions, have implications in other products. It might not be easy to predict the effects of taking a NICI action. NICI is one of the most critical services in the system, If certain NICI keys are irrecoverably lost, even backed-up data might be useless, because it can't be decrypted without the corresponding key.

All information is advisory. The contents of this document do not guarantee a fix.

1.1.1. Understanding the NICI modules

When you install NICI, the installation process creates NICI modules on your server.

NICI modules on Linux servers

The following table lists the modules that the NICI installation process places on your Linux server.

Module	Description
libccs2.so	NICI is a shared object (.so) named libccs2.so . Typically, it is a symbolic link to the actual file named per platform and version. NICI does not depend on directory services to be installed.
libniciext.so	niciext is shipped with OpenText eDirectory and provides OpenText eDirectory applications with secure communication. It also provides a shared key for common secured storage..

NICI modules on Windows servers

The following table lists the modules that the NICI installation process places on your Windows server.

Module	Description
ccswx64.dll (64-bit) ccs.dll (32-bit)	NICI is a shared library (dll) named <code>ccswx64.dll</code> or <code>ccs.dll</code>. It is typically installed in the <code>%windir%\system32</code> directory. For example, on a 64-bit box: <code>ccswx64.dll</code> : <code>c:\Windows\System32</code> <code>ccs.dll</code> : <code>c:\Windows\SysWOW64</code> On a 32-bit box: <code>ccs.dll</code> : <code>c:\Windows\System32</code> NICI does not depend on directory services to be installed.
niciextwx64.dlm	<code>niciextwx64.dlm</code> is shipped with OpenText eDirectory and provides OpenText eDirectory applications with secure communication. It also provides a shared key for common secured storage.

1.2. What's new

NICI 3.2 provides the following features:

- [Support for 8192 bit RSA Encryption](#)
- [FIPS Mode for NICI](#)
- [Operating NICI in FIPS Mode](#)
- [NICI SDI Health Check](#)
- [Creating an Inherited Rights Mask](#)
- [Dependency Change](#)

1.2.1. Support for 8192 bit RSA encryption

NICI 3.2 (64 bit) supports 8192 bit RSA encryption. For more information, see [Using 8192 Bit RSA Keys in Certificates](#) in the [OpenText eDirectory Administration Guide](#).

1.2.2. FIPS mode for NCI

NCI 3.2 (64 bit) is FIPS 140-2 Inside validated using the OpenSSL FIPS Object Module. For more information, see the [OpenSSL FIPS Object Module](#).

By default, FIPS mode for NCI is turned off, and must be turned on to operate NCI in FIPS 140-2 mode. For more information about putting NCI in FIPS mode, see [Operating NCI in FIPS Mode](#). In case of OpenText eDirectory, see [Operating OpenText eDirectory in FIPS Mode](#) in the [OpenText eDirectory Installation Guide](#).

1.2.3. Operating NCI in FIPS mode

There are 3 ways that NCI can be put into FIPS mode:

- **Application API:** A NCI enabled application can specify it wants to operate in FIPS mode by setting a parameter on the appropriate API. This gives each NCI enabled application the ability to specify that they are to operate in FIPS mode. Once an application is in FIPS mode it cannot change to a non-FIPS mode until it shuts down completely.

Process API: A NCI enabled application can specify that all applications in the same process space are to operate in FIPS mode. To enable FIPS mode, an application just need to call the appropriate API. This allows a NCI enabled application to confirm that all applications in the same process space are operating in FIPS mode. For example, a single eDirectory application can call the API resulting in all NCI enabled applications operate in FIPS mode. Once a process space is put into FIPS mode, it cannot change to a non-FIPS mode until the process shuts down completely.

Computer level via NCI Configuration File: NCI can be configured so that all 64-bit NCI applications on that computer will operate in FIPS mode. For more information, see [Using NCI for Configuring System-Level FIPS Mode](#).

The configuration is read and applied at application start-up time. Once an application is in FIPS mode it cannot change to a non-FIPS mode until it shuts down completely.

1.2.4. NICI SDI health check

In NICI 3.2, the NICI SDI module (`niciext`) now has a health check which runs each time that the `niciext` module is loaded. The health check also runs if a new key is created. The output from the health check is output a file, and can also be seen via the `DSTrace` module.

1.2.5. Creating an inherited rights mask

To fully support multiple SDI keys, the NCI health check process runs periodically and creates an inherited rights mask for the `CN=KAP.CN=Security` object. An inherited rights mask is created automatically to address a previous security right issues.

1.2.6. Dependency change

NICI 3.2 (64-bit) no longer requires 32-bit NICI to be installed. Both 32-bit and 64-bit NICI share the same key files.

1.3. Installing NCI

This section provides the prerequisites, considerations, and system setup to install NCI in your environment.

1.3.1. Checklist for installing NICI

It is recommended that you complete the steps in the following checklist:

	Checklist Items
	1. Determine which operating system platforms suit your installation. For more information, see System Requirements.
	2. Review the considerations for installing NCI. For more information, see Prerequisites and Considerations for Installing NCI.
	3. Understand how to install NCI. For more information, see Understanding the NCI Installation.
	4. Ensure access the installation files for NCI, located by default in the installation package of the products bundling NCI.
	5. Ensure that you have the appropriate permissions for installing NCI. For more information, see the following links: ◦ Installing NCI as a Root User ◦ Installing NCI as a Non-Root User With Root Access

	Checklist Items
	6. Review the actions that you can perform after installing NCI, see Post-Installation Tasks .

1.3.2. Understanding the NCI installation

Novell bundles NCI with OpenText eDirectory and OpenText Identity Console in the same installation programs. NCI is also available as part of Novell Client. However, NCI has its own installation program and can be installed separately. It is recommended that NCI be installed by the software program packaging NCI.

1.3.3. Prerequisites and considerations for installing NICI

Before installing NICI, review the following considerations:

- You must install NICI as a root user or a root-equivalent user.
- NICI is a shared library (DLL , .so). providing API services to applications. It does not run as a separate program or service.
- NICI provides additional capabilities to OpenText eDirectory via the niciext module that is shipped as part of the OpenText eDirectory distribution and that runs as an OpenText eDirectory service.
- You must install NICI on every workstation that use management utilities for OpenText eDirectory, such as OpenText Identity Console.
- NICI and OpenText eDirectory support RSA key sizes up to 8192 bits.
- You can install NICI separately or with OpenText eDirectory, OpenText Identity Console, or Novell Client on platforms supported by these products. For more information on supported platforms, see [System Requirements](#).
- Uninstalling or reinstalling NICI does not destroy existing keys (i.e. key files are not removed from the file system during uninstall or during reinstall).
- Installing NICI 3.x does not require rebooting the server in most instances. However, if the NICI module (DLL or .so) is in use and cannot be overwritten by the installation program, a reboot might be necessary. Before installing NICI, all applications which are using NICI should be shut down to help avoid reboots.
- Installing a newer version of NICI over an existing NICI installation upgrades NICI. Always upgrade NICI using the NICI installation program (MSI or RPM). Do not copy NICI modules manually. Manual copying will result in a chaotic system, and may cause irreparable damage to the system and/or other products such as PKI, SecretStore/Single Sign-On, NMAS, directory services, etc.

1.3.4. System requirements

You can install NCI separately or with OpenText eDirectory, OpenText Identity Console, or Novell Client on platforms supported by these products. For more information on which platforms you can install NCI, see the following links:

- [OpenText eDirectory](#)
- [OpenText Identity Console](#)
- [Novell Client](#)

1.3.5. Installing NCI

This section describes the process for installing NCI. Review the prerequisites and system requirements provided in [Prerequisites and Considerations for Upgrading NCI](#) and [System Requirements](#).

Installing NCI on Linux servers

By default, the NCI installation file is packaged in the installation kit of the product bundling NCI. While installing the product, select the option to install NCI. Novell recommends installing NCI as root because the required NCI packages are used system-wide. However, if necessary you can delegate access to a different account using the Sudo utility and use that account to install the NCI packages.



Note

If you are using NCI 2.7.7, upgrade to the NCI 3.x 32-bit first then to 64-bit.

Depending on your product, follow the installation instructions from one of the following links:

- **OpenText eDirectory:** [“Installing NCI”](#) in the [OpenText eDirectory Installation Guide](#)
- **OpenText Identity Console:** [Deploying Identity Console](#).
- **Novell Client:** [“Installing the Novell Client for Windows”](#) in the [Novell Client 2 SP4 for Windows Installation Quick Start](#)

If the setup program detects a previously installed version of NCI, it might give you the option to stop the installation process, remove, or upgrade the existing installation.

This section describes the following activities:

Installing NCI as a root user

To install NCI, enter the following commands from 32-bit and 64-bit NCI directories:

- **32-Bit:** `rpm -ivh NCI_rpm_absolute_path/nici-3.2.0.i586.rpm`
- **64-Bit:** `rpm -ivh NCI_rpm_absolute_path/nici64-3.2.0.x86_64.rpm`



Note

Upgrading OpenText eDirectory from 8.8 to 9.0, does not upgrade the 32-bit NCI.

Installing NCI as a non-root user with root access

To install NCI on Linux servers, you must login as `root` or have `root` access. The Sudo utility provides an easy way for a system administrator to delegate `root` access to other user to perform specific tasks by editing the `/etc/sudoers` configuration file and adding appropriate entries in it.

As with any delegation of rights, the system administrator should take the necessary precautions to ensure that the company's security standards are followed.

Most Linux providers have the Sudo utility available on their Web sites. You can also download the Sudo utility from the [Sudo Web site](#). The Sudo documentation provides information on how to install and configure the utility. When it is configured properly, the user can install NCI by entering `sudo install_command`.

The installation program might place the following files on your computer:

- `nicifk.new`
- `set_server_mode` (Linux)

NCI uses these files to switch NCI into server mode when programs such as OpenText eDirectory are installed. To verify that NCI is set to server mode, you can enter `/var/opt/novell/nici/set_server_mode64`.

Installing NCI on Windows servers

By default, the NCI installation file is packaged in the installation kit of the product bundling NCI. To install NCI on Windows, perform the following steps:

- **Installing 64-bit NCI:**
 - Double click on `NCI_wx64.msi`.
 - Click **Install**.
- **Installing 32-bit NCI:**
 - Double click on `NCI_x32.msi`.
 - Click **Install**.

1.3.6. Meeting the LSB compliant directory structure

NICI installations for Linux platforms are LSB-compliant. NICI uses the following LSB directory structure:

LSB Directory Structure

File/Directory Type	Directory
Configuration file (nici64.cfg or nici.cfg)	/etc/opt/novell
License file and user directories	/var/opt/novell/nici
Library file	/opt/novell/lib 64 (if NICI 64-bit is installed) /opt/novell/lib (if NICI 32-bit is installed)
Man pages	/opt/novell/man

[Symbolic Links](#) shows the symbolic links for Linux servers.

Symbolic Links

Platform	Symbolic Link
Linux	/opt/novell/lib/libccs2.so → /opt/novell/lib/libccs2.so.3.0.0 /opt/novell/lib64/libccs2.so → /opt/novell/lib64/libccs2.so.3.0.0

1.3.7. Locating the NICI configuration files

The installation program places the NICI configuration files in the directories listed in [NICI Configuration Directory](#).

NICI Configuration Directory

Platform	Library Location	NICI Directory	NICI User Directory
Windows	• 32-bit: %SystemRoot%\System32 • 64-bit: %SystemRoot%\SysWow64	• 32-bit: %SystemRoot%\System32\Novell\NICI • 64-bit: %SystemRoot%\SysWow64\Novell\NICI	• 32-bit: %SystemRoot%\System32\Novell\NICI • 64-bit: %SystemRoot%\SysWow64\Novell\NICI
Linux	• 32-bit: /opt/novell/lib • 64-bit: /opt/novell/lib64	• 32-bit: /var/opt/novell/nici • 64-bit: /var/opt/novell/nici	• 32-bit: /var/opt/novell/nici/ • 64-bit: /var/opt/novell/nici/



Note

NICI configuration files are shared by both 32-bit and the 64-bit NICI.

[NICI Configuration Files](#) lists additional files that the installation program places on all platforms.

NICI Configuration Files

File	Created by	Description
NICIFK	NICI installation	NICI license material for server-mode operation.
xmgrcfg.wks	NICI Install	NICI license material for client-mode operation. Not used if NICIFK is present.
xmgrcfg.nif	First use of NICI or by install by a privileged user	NICI per-box unique keying material generated locally.
xarchive.000	First use of NICI by a privileged user	NICI master archive.

NICI User Configuration Files

File	Created by	Description
xmgrcfg.ks2	First use	User-specific key materials and other configuration materials.
xmgrcfg.ks3	First use or update	User-specific state data, updated occasionally.
xarchive.001	First use or update	NICI user archive.

The NCI configuration files are signed and partially encrypted. An invalid license file (`NICIFK`) or a client license file (`xmgrcfg.wks`) renders NCI non-functional.

1.3.8. Post-installation tasks

In most configurations there is no need to perform any post-installation tasks. However, NCI can be configured to meet the policies and requirements defined by your business processes. Possible post-installation tasks include the following:



Note

Both the options for [Configuring the Settings for NCI User Directory](#) and [Using NCI for Configuring System-Level FIPS Mode](#) are optional.

Configuring the settings for NCI user directory

NCI creates a new NCI user directory the first time a user uses NCI. NCI sets the rights on each user directory when it creates the directory, so that only the user has access to it.

The default directory for new NCI user directories is:

- **Linux:** `/var/opt/novell/nici/<uid of user>`
- **32-bit Windows:** `Windows\System32\Novell\NCI\<user>`
- **64-bit Windows:** `Windows\SysWOW64\Novell\NCI\<user>`

Changing the permission of NCI user directory on Linux servers

The installation program places `nici64.cfg` (64-bit NCI) and `nici.cfg` (32-bit NCI) configuration files in the `/etc/opt/novell` directory on your Linux operating system.

The NCI configuration file emulates the Windows registry and is a minimally editable text file. Most of the entries in the file are set up when you install NCI and should not be modified. Modifications to some fields will leave NCI inoperable. The entries in the configuration file vary for a 32-bit NCI and 64-bit NCI. Nevertheless, a typical configuration file contains the following entries:

```
ConfigDirectory:s:20:/var/opt/novell/nici
SharedLibrary:s:19:/opt/novell/lib64/libccs2.so
DAC:b:8:1a:aa:6d:49:48:a8:83:98
MkUserDir:s:24:/var/opt/novell/nici/nicimud
NiciVersion:s:5:2.4.0
BuildDate:s:6:020123
NiciStrength:s:2:u0
RestrictionLevel:b:1:00
```



Note

For FIPS mode, only modify the last digit of the RestrictionLevel entry.

Each line can have multiple entries all separated by a colon (:). The first entry in a line is the name, followed by its type. The second is the length in decimal, followed by the actual value. There are two types, string (s) and binary (b). For example, the name of the first line in the sample above is `ConfigDirectory`, of type string (s) 20 characters. The value is `/var/opt/novell/nici`. Each line is described in [Linux Key Values](#).

Linux Key Values

Key	Description
MkUserDir	NICI uses this executable to create user directories. /var/novell/nici/nicimud is supplied by the NICI installation program. (Do NOT modify)
NICIVersion	NICI version string. (Do NOT modify)
BuildDate	NICI module's build date; year, month, and day, each in two decimal digits. (Do NOT modify)
NiciStrength	u0 for strong, w1 for import restricted (no longer supported). (Do NOT modify)
RestrictionLevel	0 for no restriction, 1 for FIPS mode. (Modify only the last digit)
NICISDI Sync Period	(Optional) NICISDI synchronization period in minutes, represented in hexadecimal. (Not recommended)

The libniciext.so module reads the NICISDI sync period value when OpenText eDirectory loads it. If the value does not exist, or if the period is zero, the module uses an automatic sync period based on a sliding scale that starts with a heavy synchronization and moves towards lighter synchronization. If the value exists and contains a non-zero period, libniciext.so reads the value and uses it to determine synchronization periods.



Note

You should not use the optional sync period unless support directs you to do so.

The `/var/opt/novell/nici/uid/nicisdi.key` file contains the encrypted security domain keys as discussed in [Understanding the NCI Keys](#). For example, it is typically 0 for root. Having a `nicisdi.key` file for each user enables multiple instances of OpenText eDirectory running with different user IDs to host multiple trees on the same physical box.



Note

The UID is the variable numeric user ID defined by the Linux system.

All users have read and execute (where applicable) rights to the files in the NCI configuration directory (`/var/opt/novell/nici`). Only the user who installs NCI has full rights to the configuration directory. The setuid executable (`nicimud` for 32-bit NCI and `nicimud64` for 64-bit NCI), creates the NCI directories for users. For example, `nicimud` will create a directory when a user first uses NCI and will give full rights to only the user creating the directory (0700).

Changing the permission of NCI user directory on Windows servers

The NCI installation program creates and populates a key in the Windows registry. The registry key is different for 32-bit and 64-bit.

- 64-bit registry key: `HKEY_LOCAL_MACHINE\SOFTWARE\Novell\nici_x64` .
- 32-bit registry key:
 - On 32-bit machine: `HKEY_LOCAL_MACHINE\SOFTWARE\Novell\NCI`
 - On 64-bit machine: `HKEY_LOCAL_MACHINE\SOFTWARE\Wow6432Node\Novell\NCI`

Windows Key Values

Key	Type	Description
ConfigDirectory	String	Location of NCI configuration files
DAC	Binary	NCI module's digital authentication code
SharedLibrary	String	The name of the library, such as ccsw32.dll
UserDirectoryRoot	String	(Optional). Name of a directory where user directories are created. Defaults to ConfigDirectory
Version	DWORD	NCI version, such as 0x00002400 for 2.4
NICISDI Sync Period	DWORD	(Optional - Not Recommended) - NICISDI synchronization period in minutes, represented in hexadecimal.

Key	Type	Description
EnableUserProfileDirectory	DWORD	(Optional) - NCI user files are created in the Application Data\Novell\NCI directory in the user's profile directory.
RestrictionLevel	Binary	(Optional 64-bit only) - 0 for no restriction, 1 for FIPS mode.

- By default, NCI creates users directories in the %SystemRoot%\System32\Novell\NCI (32-bit) and %SystemRoot%\SysWOW64\Novell\NCI (64-bit) directories by the user's name. For example, c:\windows\sysWOW64\novell\nci\administrator .
- If you want to change the root directory in which all user directories are created, navigate to the NCI registry key and create a String value using UserDirectoryRoot as the name and the desired root directory as the value.
- When creating a user directory, NCI uses the name of the user. If it is a local user, NCI uses the username. If it is a remote or a domain user, NCI forms the username as the combination of username and domain separated by a dot (userName.domainName).
- By default the EnableUserProfileDirectory key is not created and User Profile functionality is disabled. If you enable user profile functionality, you might need to copy or move the existing NCI user files to the new location. If the user profile directory is enabled, NCI does not set the ACLs on this directory, but relies on existing security properties (ACLs, inheritance, and ownership) of the user's profile directory. Use this option very carefully, because you can disclose all users' NCI keys.
- NCI creates the Application\Novell\NCI directory if it is not present on your server and stores all NCI user files in this directory. NCI provides this option to support the dynamic user creation/deletion feature in the Novell ZENWorks®

product. It must be set manually or by another application's installation, such as ZENWorks.

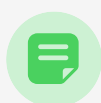
- `niciext.dlm` reads the `nici` sync period value when OpenText eDirectory loads it. If the value does not exist, or if the period is zero, the module does not attempt to read it again. If the value exists and contains a non-zero period, the value is read once in a period before synchronization. We recommend that you do not set the sync period unless directed to by support.
- The `nici`.key file contains encrypted security domain keys as discussed in [Understanding the NCI Keys](#).
- All users have read, execute, and create rights to the files in the NCI configuration directory (`<SystemRoot> \Novell\NCI`). NCI dynamically creates user directories when a user uses NCI for the first time and provides full rights only to the user creating the directory.

Using NCI for configuring system-level FIPS mode

NCI 3.2 provides the ability to turn on FIPS mode at the computer level. When FIPS mode is turned on, all 64-bit NCI enabled applications, products, and services running on that computer will be able to perform cryptographic operations only using FIPS compliant algorithms. Any attempt to use a non-FIPS compliant algorithm will fail.

To enable the FIPS mode at the computer level, perform the following steps:

- **Linux:** Navigate to the `nici64.cfg` file and change **RestrictionLevel** to 1. This file is located in `/etc/opt/novell` on Linux.



Note

Modify only the last digit of the Restriction Level setting from 00 to 01. Restart the OpenText eDirectory.

- **Windows:** Navigate to the `HKLM\SOFTWARE\Novell\nici_x64` registry and change the restriction Level settings from 0 to 1.

You will need to perform this action on each server in your tree that you wish to set each server into FIPS mode. Once done, restart the OpenText eDirectory.

**Important**

Enabling FIPS mode on NCI affects all applications that use NCI on that server. If these applications are not supported in FIPS mode, they might not work properly. Novell recommends that you do not use FIPS mode for NCI in OpenText eDirectory 9.1 to 9.2.6 versions. It supports from OpenText eDirectory 9.2.7.

1.4. Upgrading NCI

This section provides information to help you prepare for upgrading your NCI to the latest version. To install or upgrade the NCI, log in as root or a root-equivalent on the computer where you want to install or upgrade NCI.

1.4.1. Checklist for upgrading NCI

To perform the upgrade, it is recommended that you complete the steps in the following checklist:

	Checklist Items
	1. Learn about version upgrades and compatibility of versions. For more information, see Prerequisites and Considerations for Upgrading NCI.
	2. Ensure that you have the latest installation kit to upgrade NCI. For more information, see Installing NCI.
	3. Ensure that your computer meets the hardware and software prerequisites for a newer version of NCI. For more information, see Prerequisites and Considerations for Upgrading NCI.
	4. Back up the keys and user data in the file system and in system-specific and user-specific directories and files. For more information, see Backing Up and Restoring the Current Configuration.

	Checklist Items
	5. Install or upgrade NCI to the latest version For more information, see one of the following sections: ◦ OpenText eDirectory: Installing NCI in the OpenText eDirectory Installation Guide ◦ OpenText Identity Console: Deploying OpenText Identity Console.

1.4.2. Prerequisites and considerations for upgrading NCI

Before upgrading NCI, review the following considerations:

- Reinstalling NCI does not destroy existing keys.
- Installing NCI 3.x does not require rebooting the server in most instances. However, if the NCI module (DLL or .so) is in use and cannot be overwritten by the installation program, a reboot might be necessary. Before installing NCI, all applications using NCI should be shut down to help avoid reboots.
- Installing a newer version of NCI over an existing NCI installation upgrades NCI. Always upgrade NCI using the NCI installation program (MSI or RPM). Do not copy NCI modules manually. Manual copying will result in a chaotic system, and will often cause irreparable damage to the system and/or other products such as PKI, SecretStore/Single Sign-On, NMA, directory services, etc.

1.4.3. Considerations for upgrading NCI on servers with multiple instances of OpenText eDirectory

If a Linux server is hosting more than one OpenText eDirectory, each OpenText eDirectory instance typically has its own NCI directory setup. If you upgrade one instance of OpenText eDirectory to 9.0 or later, you will need to upgrade all instances because OpenText eDirectory 8.x will not work correctly using NCI 3.x.

It is recommended to run each instance of OpenText eDirectory on the same host with different user IDs to separate their cryptographic materials using the host system's security mechanisms. NCI does not require a special user to run, except for installation, when a privileged user who can install setuid programs must install NCI (a one-time operation).

1.4.4. Backing up and restoring the current configuration

For information on backing up and restoring NCI, see [Backing Up and Restoring NCI](#).

1.5. Understanding the NCI keys

To help applications securely store and transfer data and keys, NCI provides three types of keys - Key Storage key, NCI Security Domain Infrastructure (SDI) key, and Session key (SASDFM).

- The Key Storage key is a server specific key. This key is unique to the server it is created on, and is intended to be used to securely wrap keys for either internal or external storage. NCI creates this key for the server on which NCI is installed.
- A NCI SDI key is shared by all the servers within a security domain. In OpenText eDirectory a security domain consisting of the whole tree has been established and the associated key is often referred to as the Tree key or sometimes the W0 key (as the object used to manage this key is CN=W0.CN=KAP.CN=Security). In NCI 3.x and OpenText eDirectory 9.x, we have added support for a new AES-256 bit tree key (or W1 key). However, this key is not enabled by default because all servers in the tree must be OpenText eDirectory 9.x or later to support it. All the servers in an OpenText eDirectory tree have the rights to acquire the Tree key.

Access to SDI keys is governed by OpenText eDirectory rights and attributes. There is a specific set of rights and attributes that allow a server to create and distribute an SDI key. A server with this set of rights and attributes is known as a "Key server". There is a different set of rights and attributes that allows a server to acquire keys from a Key server.

- NCI provides a Session key (or SASDFM key) to securely communicate between client and server.

1.5.1. Types of keys

Understanding the key storage key

The server storage key is a computer-specific key. Each server creates a server storage key which is unique to that server and which can be used to securely wrap other keys for either local or remote storage. After a key is wrapped with a server storage key, only code on that server can unwrap the key, this allows the wrapped key to remain secure even when stored remotely.

Prior to NCI 3.x, the storage keys were Triple Data Encryption algorithm (3DES) keys. NCI 3.x creates AES 256-bit storage keys. Any application that uses the storage keys to securely wrap other keys should be able to handle the new algorithm to encrypt new data. However, any data which is currently wrapped with the older 3DES keys will still be assessable without any changes.

Understanding the session key

To securely send data from a client to a server or server to server or vice versa, NCI provides the SASDFM keys which serve as a session key. Prior to NCI 3.x, the session keys were 3DES keys. NCI 3.x supports AES 256-bit session keys, as well as 3DES keys, depending on the capabilities of the applications.

- The client application and the OpenText eDirectory server will use the AES 256-bit session key only if both of them use NCI 3.x.
- If either of them uses a lower version of NCI, they will use a 3DES session key.

Understanding the NCI SDI key

NCI SDI (Security Domain Infrastructure) is an OpenText eDirectory service which provides and manages shared keys for all servers within a security domain. Access to SDI keys is governed by OpenText eDirectory rights and attributes. There is a specific set of rights and attributes that allow a server to create and distribute an SDI key. A server with this set of rights and attributes is known as a **Key server**. There is a different set of rights and attributes that allows a server to acquire keys from a Key server.

NCI SDI can manage multiple keys of varying strengths and algorithms. Each SDI key can have a different security domain and is controlled by the OpenText eDirectory rights and attributes of the OpenText eDirectory object representing the SDI key known as the SDI key object:

- **Linux:** libniciext.so
- **Windows:** niciext64.dlm

The security domain keys are not intended for clients.

Understanding tree keys

Tree keys are a special kind of NCI SDI key. The security domain for tree keys consists of the whole OpenText eDirectory tree, and they are automatically managed by OpenText eDirectory and NCI SDI.

In all OpenText eDirectory versions prior to 9.0, a single security domain consisting of the whole tree has been established and the associated key is often referred to as the Tree key or sometimes the `W0` key (as the SDI key object used to manage this key is `CN=W0.CN=KAP.CN=Security`). This key is a `3DES` key, and all the servers in an OpenText eDirectory tree have the rights to acquire this key. This key will continue to be available.

Beginning in OpenText eDirectory 9.0 with NCI 3.0, OpenText eDirectory supports the creation of a new AES 256-bit Tree key. The SDI key object used to manage this new Tree key is `CN=W1.CN=KAP.CN=Security`. It is required that all servers in the tree be upgraded to OpenText eDirectory 9.x before enabling this key. Although OpenText eDirectory 9.x will automatically create this SDI key object, it will not assign a `Key server` and the key will not get created by default. An administrator will need to assign a Key server to the SDI key object, after confirming that all servers in the tree have been upgraded to OpenText eDirectory 9.x, in order to enable the new AES 256-bit Tree key. For more information, see [Creating an AES 256-Bit Tree Key](#).

Although any server can be configured as a Key server for the tree keys, it is recommended that only servers holding a writeable replica of the SDI key object be assigned. It is recommended that the first Key server assigned be the Master replica (for example, the server holding the Master replica of the object `CN=W1.CN=KAP.CN=Security`).

NCI SDI supports having multiple Key servers for any SDI key and it is recommended that multiple Key servers be assigned. In NCI 3.x, once a Key server has been assigned to the Tree key objects, the new Heath-Check feature will automatically add servers holding a writeable replica of the SDI key object). The idea here is that NCI SDI will automatically mirror the Key servers to your OpenText eDirectory replicas.

Various services rely on the availability of Tree keys, including but not limited to SecretStore/Single-Sign-On, PKI (Certificate Server), and NMAS.



Note

The NICISDI module is different from the SASDFM module. SASDFM manages session keys between two physical boxes, typically between a client and a server.

1.5.2. Managing the tree keys

Tree keys are a special kind of NCI SDI key. Like all SDI keys, tree keys are managed by the associated SDI key object and its attributes. There are now two Tree key objects, `CN=W0.CN=KAP.CN=Security` which manages the older 3DES Tree key (or the W0 key), and `CN=W1.CN=KAP.CN=Security` which manages the new AES 256-bit Tree key (or the W1 key).

The new object `CN=W1.CN=KAP.CN=Security` will get created when a server holding a writable replica of the `CN=KAP.CN=Security` container is upgraded to OpenText eDirectory 9.x. Although OpenText eDirectory 9.x automatically creates this SDI key object, it will not assign a Key server and the key will not get created by default. An administrator will need to assign a Key server to the SDI key object, after confirming that all servers in the tree have been upgraded to OpenText eDirectory 9.x, in order to enable the new AES 256-bit Tree key. For more information, see [Creating an AES 256-Bit Tree Key](#).



Note

Do not delete the W0 object after upgrading the tree.

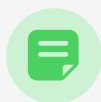
The W0 and W1 objects contain the following attributes:

NDSPKI: SD key server DN

The NDSPKI:SD Key Server DN attribute is used to determine which servers are Key servers and can create and distribute the SDI key.

This multivalued attribute on the W0 or W1 objects contains the list of the key servers in the tree for the respective SDI key object. There must be at least one server in this list for the SDI key object to be active. The `niciext` module reads this attribute and then connects to each server in this list and requests any new security domain keys from each server in this list. Only servers in this list can create and distribute the tree key.

Adding a server to this attribute makes that server a Key server. Although any server can be configured as a "Key server", for the tree keys, it is recommended that only servers holding a writeable replica of the SDI key object be configured.



Note

If a key server does not hold a writeable replica, additional rights will need to be assigned.

The OpenText eDirectory Install will automatically populate this attribute for the W0 object, so no action is required by an administrator for the W0 object. For the W1 object, an administrator will need to assign a Key server to this attribute, after confirming that all servers in the tree have been upgraded to OpenText eDirectory 9.x, in order to enable the new AES 256-bit Tree key. It is recommended that the first Key server assigned be the Master replica (for example, the server holding the Master replica of the object `CN=W1.CN=KAP.CN=Security`).

ACL

The ACL attribute is used to determine which servers can acquire a copy of the key. Servers need the Read right to the [All Attributes Rights] attribute and the Browse rights to the object in order to be able to acquire a copy of the key.

In 3.x, the NICI SDI health check process runs periodically and creates an inherited rights mask for the KAP.Security object. The inherited rights mask is created automatically to make sure only servers which have been specifically granted rights to a SDI key object are able to acquire the key (objects are not allowed to inherit the rights necessary to acquire an key, they must be specifically assigned the rights).

1.5.3. Synchronizing the NICI SDI keys

The NICI SDI module (`niciext`) automatically syncs the NICI SDI keys. Normally the module uses an automatic sync period based on a sliding scale that starts with a heavy synchronization and moves towards lighter synchronization.

If a new SDI key is enabled or created, you can speed the synchronization process to each of the servers in the tree by using one of the following methods on each server in the tree:

- **Linux:** Unload and reload the NICI SDI module (`niciext`) using `ndstrace` .
- **Windows:** Use the DHost console to reload and `niciext` module.
- Restarting OpenText eDirectory.
- Rebooting the server.

There is an optional (not recommended) configuration setting that can adjust the synchronization period. See [Configuring the Settings for NICI User Directory](#) for more details.



Note

You should not use the optional sync period unless support directs you to do so.

1.5.4. Creating an AES 256-Bit tree key

Tree keys are a special kind of NCI SDI key and are available to all servers in the tree. When multiple servers need access to the same encrypted data, OpenText eDirectory uses the Tree keys to provide access while still keeping the data secure in conjunction with OpenText eDirectory rights. In all prior versions of OpenText eDirectory a single security domain consisting of the whole tree has been established and the associated key is often referred to as the Tree key or sometimes the W0 key (as the SDI key object used to manage this key is `CN=W0.CN=KAP.CN=Security`). This key is a 3DES key, and all the servers in an OpenText eDirectory tree have the rights to acquire this key. This key will continue to be available.

Beginning in OpenText eDirectory 9.0 with NCI 3.0, it supports the creation of a new AES 256-bit Tree key. The SDI key object used to manage this new Tree key is `CN=W1.CN=KAP.CN=Security`. This key will be known as the W1 key. It is required that all servers in the tree be upgraded to OpenText eDirectory 9.x before enabling this key. Although OpenText eDirectory 9.x will automatically create this SDI key object, it will not assign a Key server and the key will not get created by default. An administrator will need to assign a Key server to the SDI key object, after confirming that all servers in the tree have been upgraded to OpenText eDirectory 9.x, in order to enable the new AES 256-bit Tree key.



Important

- Do not create an AES 256-bit key unless all servers in your tree are upgraded to 9.x.
- Creating an AES 256-bit key with OpenText Identity Manager causes all passwords to be re-synced. For more information, see [Re-encrypting Data with AES 256-Bit NCI SDI Key](#) in the [OpenText eDirectory Administration Guide](#).

When a server holding the master replica of the KAP.Security container is upgraded to OpenText eDirectory 9.x, OpenText eDirectory install creates a W1 object in this container. When all servers in the tree are upgraded to OpenText eDirectory 9.x, the tree administrator can create an AES 256-bit SDI key

1. Log in to the OpenText Identity Console as an administrator with the appropriate rights and goto **Tree View**.
2. Goto **Security Container** under tree view.

3. Click **KAP**.
4. Browse and select the **W1** security object.
5. Click **OK**.
6. In the window that displays, add NDSPKI:SD Key Server DN attribute and set the value to the DN of a server holding a master replica of the partition that contains the W1.KAP.Security object.
7. To create the AES 256-bit SDI key, trigger the NICI health check by performing one of the following actions:
 - **Linux:** Unload and reload the NICI SDI module (`niciext`) using `ndstrace`.
 - **Windows:** Use the DHost console to reload and `niciext` module.
 - Restart OpenText eDirectory.
 - Restart server.

After the AES 256-bit SDI key is created, the new key will automatically be synchronized to all servers in the tree using the normal synchronization schedule. If the servers in the tree have been up for some time, the automatic synchronization process is likely to be slow because SDI keys are synchronized on a sliding scale depending on how long the SDI module has been running. You can speed the synchronization process to each of the servers in the tree by using one of the following methods on each server in the tree:

- **Linux:** Unload and reload the NICI SDI module (`niciext`) using `ndstrace`.
- **Windows:** Use the DHost console to reload and `niciext` module.
- Restart OpenText eDirectory.
- Restart server.



Important

The NICI SDI key is available to all servers in the tree. Therefore, you must upgrade all servers in the tree to NICI 3.0 before creating the AES 256-bit SDI key.

1.5.5. NICI SDI health check

In NICI 3.x, the NICI SDI module (`niciext`) now has a health check which runs each time that the `niciext` module is loaded. The health check also runs if a new key is created. The output from the health check is output to `NIClExt_Health.log` file located in the normal OpenText eDirectory log directory. In addition, the output can be seen in `DSTrace` , if enabled.

The NICI SDI health check process performs the following tasks:

- Creates an inherited rights mask for the KAP.Security object.



Note

The inherited rights mask is created automatically to address a security rights issue.

- Automatically adds servers with a Writable replica of the W0 object to be key servers for the W0 object.
- Automatically creates a new W1.KAP.Security object. NICI uses this object to represent and administer rights to the new AES 256-bit SDI key.



Important

NICI does not automatically create the new AES 256-bit SDI key until a tree administrator performs a specific configuration operation.

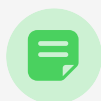
- Checks to see if a key server has been assigned to the W1 object. Only if a key server has been assigned, the NICI health Check utility will add servers with a Writable replica of W1 object to be Key Servers for W1. For more information, see [Creating an AES 256-Bit Tree Key](#).
- Checks to see if a key server has been assigned to the W1 object. Only if a key server has been assigned, the NICI health check will mirror the rights for the W0 object to the new W1 object, which will allow all servers in the tree to get access to the new AES 256-bit SDI key.

1.5.6. Merging trees

In order for a tree merge to be successful, all servers will need to have a copy of the tree key(s) from all the original trees.

Before merging, you should identify the key servers for each of the tree keys (W0 and W1) in all the trees.

After merging, for each of the tree keys (W0 and W1) you should add the names of all the SD key servers to the corresponding object from all the merged trees. For each object (W0 and W1), the final list must contain the names of the SD key servers, for that object, from all the trees being merged.



Note

If either of the trees have been created in AES 256-bit tree key, then all servers in both trees **MUST** be upgraded to OpenText eDirectory 9.x before merging.

1.5.7. Diagnosing SDI key synchronization issues

The addition of the NICI SDI Health check will minimize synchronization issues. Output from the health check will help to identify any issues that can't be automatically fixed by the health check. In addition, more detailed synchronization messages can be seen and captured in `DSTrace`.

Regardless of the platform/OS, the `nicisdi.key` file is server-unique and should not be copied from one machine to another. Manual creation of a new key typically causes more problems by introducing a new key on the server.

With the advent of the SDI Health check, NICI SDI is designed to fix itself. However, we will continue to provide `SDIDiag` as a Security Domain Infrastructure diagnostic and repair utility. Among other things, `SDIDiag` allows an administrator to:

- Run `CHECK` to verify that all Security Domain servers have a consistent key set
- View the various keys within an OpenText eDirectory container or tree
- Ensure that all servers are synchronized with consistent keys

For information about using this utility, see [TID #319224010081773](#).



Important

If you have installed OpenText eDirectory to use a non-standard port, you must specify the port number with the IP address when you run `SDIDiag`. For example, `xxx.xxx.xxx.xxx:port`.

1.6. Backing up and restoring NICI

NICI stores keys and user data in the file system and in system-specific and user-specific directories and files. The NICI installation program protects these directories and files by setting the proper permissions on them, using the mechanisms provided by the operating system.

Uninstalling NICI from the system does not remove these directories and files; therefore, the only reason to restore these files to a previous state is to recover from a catastrophic system failure or a human error. Also, overwriting an existing set of NICI user directories and files might break an existing application.

Backing up and restoring NICI requires two things:

1. Backing up and restoring directories and files
2. Backing up and restoring specific user rights on those directories and files

The exact sequence of events required is platform-dependent.

When you back up and restore NICI, it is critical that you maintain the exact permissions on the directories and files. NICI's operation and the security it provides depends on these permissions being set properly.

Typical commercial backup software should preserve permissions on the NICI system and user directories and files. You should check your commercial backup software to see if it does the job before you do a custom backup of NICI.

You should always back up the existing NICI directory structure and its contents, if any, before doing a restore. If you lose the machine key, it is unrecoverable. Because the user data and keys could be encrypted by using the machine key, losing it results in a permanent loss of user data.

To do a restore of NICI only, you must understand which specific files must be restored. During restoration, it is important that the correct access rights be restored for the correct owner. On Linux and Windows systems, the name of the user-specific directory reflects the ID of the owner, but on both systems the owner ID might change between the time of the backup and the time of the restore. It is important for security reasons that you know which account is being restored and that you assign the directory name and access rights accordingly. The mere existence of a user account on the system with the same ID as what was backed up does not mean that the current account is the actual owner of the information being restored.

- [Performing a Backup](#)

- [Restoring NCI](#)
- [Automatic Backing Up and Restoring NCI](#)

1.6.1. Performing a backup

Applications that use NCI to perform cryptography might have dependencies on data that NCI manages. If so, it might be necessary to back up the NCI configurations files in order to recover the encrypted data, or just to preserve the state of the files as part of an incremental backup. This section assumes that you have other means to perform disaster recovery or rebuild a system and just need to know which files must be backed up and restored in order to preserve critical NCI data that is not recoverable by simply reinstalling NCI. You should consult the individual application documentation to determine if NCI data is critical to the application. If it is, the NCI files should be backed up at the time the application data is backed up.

Performing a backup on Linux systems

In NCI 3.x, the `/var/opt/novell/nci` directory contains all the system and user directories and files.

Most NCI configuration files are located in the `var/opt/novell/nci` directory. A few NCI configuration files are located in the `/etc/opt/novell` directory. The configuration files are associated with each user account on the operating system. In order to back up a user's configuration files, you must preserve the contents of the novell configuration directory located in the following tables and the user-specific subdirectory within it (alternatively, back up everything within the directory). You might find some executables in the directory. They do not need to be backed up.

Some of the critical NCI configuration files listed in [NCI Configuration Files](#) are unique to a specific user. Most configuration files are contained within the `/var/opt/novell/nci` directory, which contains common files. Files unique to specific users are contained within subdirectories of this directory. For simplicity, you can back up the entire directory structure or back up the common files and specific user files, whichever is most convenient. Be sure that you can restore the access rights on the directories and files later. When you restore the files you can make decisions about exactly which files must be recovered. Be sure to note which version of NCI is installed at the time, because the configuration files may not be compatible with earlier versions.

You must back up the directories and files listed in [Directories and Files for Backup](#). Remember to preserve the rights on all the directories and files.

Directories and Files for Backup

Directory/File Name	File Type and Special Instructions
/etc/opt/novell/nici.cfg (32-bit) /etc/opt/novell/nici64.cfg (64-bit)	Configuration file.
/opt/novell/lib/libccs2.so .* or /opt/novell/lib64/libccs2.so*	NICI library. The version of the library completes the name.
/var/opt/novell/nici	Contains all the system keys, user directories, and files, and the programs used to initialize NICI.

On servers where you installed OpenText eDirectory, you can backup the NICI keys using the DSBK utility as instructed in [Backing Up NICI](#) in the [OpenText eDirectory eDirectory Administration Guide](#).



Note

Depending on your operating system and the version of NICI installed, there might be additional files, particularly executable files, within the directories. Those additional files, which are created during NICI installation, do not need to be backed up. See [NICI Configuration Files](#) for a list of the configuration files.

Performing a backup on Windows servers

Configuration information is kept in the Windows registry. The registry key is different for 32-bit and 64-bit:

- **64-bit registry key:** `HKEY_LOCAL_MACHINE\SOFTWARE\Novell\nici_x64`
- **32-bit registry key:**
 - On 32-bit OS: `HKEY_LOCAL_MACHINE\SOFTWARE\Novell\NICI`
 - On 64-bit OS: `HKEY_LOCAL_MACHINE\SOFTWARE\Wow6432Node\Novell\NICI`

1. Backup all registry information under the appropriate register key(s) for your installation. If you have both 32-bit and 64-bit NCI installed you will need to backup both registry keys.
2. Backup the directory, including subdirectories, identified by

[Your registry key]\ConfigDirectory

Both 32-bit and 64-bit NCI share the same config directory, so if both are installed, this value will be the same for both and you will only need to backup one directory.

On servers where you installed OpenText eDirectory, you can backup the NCI keys using the DSBK utility as instructed in [Backing Up NCI](#) in the [OpenText eDirectory Administration Guide](#).

If commercial software is used to do the backup, make sure the backup program itself runs as a system process. This ensures that the program can access all the directories and subdirectories.



Note

- Novell recommends to backup NCI using `dsbk` with the `-e` option to ensure that NCI is not in use while performing the backup.
- If `dsbk` is not used, at least OpenText eDirectory should be shutdown before performing the backup.

1.6.2. Restoring NCI

At some point it might be necessary to recover NCI configuration files so that the information they contain can be used to decrypt data for an application or simply to restore NCI to a previous state. We assume that you backed up the NCI configuration files at the same time you backed up the application.



Caution

Overwriting existing NCI configuration files can cause critical data to be lost. If an application has used NCI to encrypt data and the NCI configuration files are lost, it might not be possible to recover the encrypted data. Always keep copies of any files you overwrite. Different applications might have conflicting needs and you might need to recover the data for one application, then restore the system again to recover the data for a second application or continue with normal operations.

Restoring NCI on Linux systems

1. Reinstall NCI to a known good state.
2. Determine which user files must be restored.

It might be necessary to recover files from one user directory and place them in a different user directory if the users on the system have changed. For example, if Bob originally encrypted data, then the data should not accidentally be revealed to Mary.

3. Recover the common configuration files and the appropriate user-specific files.

This may invalidate the configuration files for other users not recovered from the same backup. It might be appropriate to just delete all the configuration files before attempting to restore any specific user files. Re-establish the correct access rights so that each user has approved access to the correct configuration files.

4. On server where you installed OpenText eDirectory, you can restore the NCI keys using the DSBK utility as instructed in [Restoring NCI](#) in the [OpenText eDirectory Administration Guide](#).

The administrator should follow the above steps. But a knowledgeable operator might choose to restore individual files or directories, possibly changing the names of the files or directories and assigning new access rights.

This can be done if the `nicifk` and `xmgrcfg.wks` files haven't changed from those on the backup store.

Review the following guidelines for each file/directory before restoring NCI if NCI is already installed on the server:

Filename	Guidelines
<code>xarchive.000</code>	Can be restored over an existing file.
<code>xmgrcfg.nif</code>	Can be restored over an existing file.
User-specific directories and files	Make sure that the user ID in the backup is the same as the user on the box. If the user directory already exists, then it must be determined if the user wants to keep the current files or restore them to a previous state. Normally, user configuration files should be restored as a group rather than individually. Be sure to restore the user files under the correct user's user ID and to restore the rights on the user directory and contents. For example, if BOB had user ID 1000 at the time of the backup but now has user ID 5000, then the files in the backed up directory 1000 should be restored to directory 5000, or BOB's UID must be changed back to 1000. So, the restore process must not just blindly restore the user directories without input from the operator. In either case, a backup of the existing NCI user directory needs to be done.

Restoring NCI on Windows servers

1. Determine if NCI is already installed on the server by searching the registry for the NCI registry keys mentioned in [Performing a Backup on Windows Servers](#), then do one of the following:
 - If NCI is not installed, restore all the registry information first.
 - If NCI is installed, remove NCI and overwrite the registry information from the backup store.
2. Restore the files and directories within `[You registry key]\ConfigDirectory` as selected by the operator.
3. On server where you installed OpenText eDirectory, you can restore the NCI keys using the DSBK utility as instructed in [Restoring NCI](#) in the [OpenText eDirectory Administration Guide](#).

It is recommended that all the files be restored as a group. But if you are knowledgeable, you can choose to restore individual entries. This can be done only if the `nicifk` and `xmgrcfg.wks` files did not change from the files in the backup store. If this is the case, be sure to adjust the access rights based on the new owner of the user configuration directories. The individual directories are named after the owner, but access rights are controlled by the SID. For example, just because a subdirectory is named BOB does not automatically mean that the current user BOB is the correct owner of the information being restored.

Special cases for Windows

It is possible to configure the registry value `HKEY_LOCAL_MACHINE\SOFTWARE\Novell\NCI\UserDirectoryRoot` 32-bit or `HKEY_LOCAL_MACHINE\SOFTWARE\Novell\ nci_x64\UserDirectoryRoot` 64-bit to indicate that the user configuration files are to be placed in the user's personal configuration directory. In this case, you should be prepared to back up and restore the user information independently as part of normal backup and restore operations. If NCI has been configured in this manner, you should be aware of it and be prepared to do individual backups.

This special case for the Windows user directory is enabled by creating the registry value `EnableUserProfileDirectory` rather than just pointing the directory path there. When the user profile directory is enabled, the directory might be automatically deleted when Windows is configured to automatically create and delete user accounts. In this case, backup and restore is necessary only for those specific users who are permanent.

The default path is the `Application Data\Novell\Nici` directory branch of the user's directory in Documents and Settings.

1.6.3. Automatic backing up and restoring NICI

In order to make NICI as stable and self-healing as possible, NICI creates a local backup every time it is successfully ran. The advantage of this is NICI can often automatically detect and recover in the unlikely event of data corruption. The fact that NICI configuration and data files are generally fairly static, means that in most cases no data will be lost even if an automatic recovery is needed.

Although the fact that NICI creates a local backup will eliminate most data corruption issues, it is recommended that you continue to regularly perform an off-line backup of NICI.

1.7. Resolving errors

This section provides NCI error messages and information on how to resolve the errors.

1.7.1. Error messages

Error -1473: NICI E-OpenSSL Failure

This is an error generated by the FIPS OpenSSL libraries that NICI uses.

At this time, we do not have any specific workaround for this error code. Contact support for more details.

Error -1460: NICI_E_NOT_FOUND

If returned when trying to initialize NICI on a Windows platform, this error typically means that NICI is not installed.

This error is returned when a security domain key (such as a tree key) is not found on the system. The API is CCS_GetPartitionKey. See [Understanding the NICI Keys](#) for more information.

Error -1470: NICI_E_FIPS140CNRG_ERR

This is an error in NICI's internal random number generator as defined by FIPS 140. NICI will try to recover, and returns this error if it can't. The solution is to retry, reload, or restart the application. For more details, contact support.

Error -1471: NICI_E_SELF_VERIFICATION

This error condition was introduced with the FIPS 140-certified NICI. Upon loading or being instantiated by a process, NICI runs a set of tests for module integrity as well as cryptographic process integrity. If one of these tests fails, NICI puts itself in an inoperable state and returns this error. The typical cause of this problem is module verification failure. The solution is to reinstall NICI, or to uninstall and then reinstall NICI.

Error -1472: NICI_E_CRYPTO_DOWNGRADE

This error was introduced in NICI version 2.0.1. The most likely cause is installation of a weak NICI version on a strong NICI installed base. The solution is to install strong NICI.

OpenText is shipping the strong NICI worldwide, and stopped shipping the import-restricted version with limited key sizes.

Error -1494: NICI_E_NOT_INITIALIZED

Similar to error -1497, this is typically caused by the lack of NCI license materials or configuration files. Reinstalling NCI typically solves the problem. If it does not, first try removing the NCI registry key on Microsoft Windows, deleting the Linux `/etc/nici.cfg` configuration file, and then installing NCI. Reinstalling NCI does not remove existing keys. If this doesn't solve the problem and you won't lose data by deleting the NCI configuration files and keys, then delete the NCI configuration directory together with the registry on Microsoft Windows or the Linux configuration file, and reinstall NCI.

Error -1497: CCS_E_AUTHENTICATION_FAILURE

Typical causes:

- Lack of NCI licensing materials (`.nfk` file copied to the `nicifk` file). NCI on servers (DHost, or equivalent environment on other platforms) must have a NCI foundation key file in order to initialize key materials. NCI license materials are part of a OpenText eDirectory license. NCI does not operate without a NCI licensing materials, or a proper configuration file. The solution is to install a license (this can be the installation of the same license), or copy the `.nfk` file from the license diskette to the `nicifk` file, then reboot the server or restart the DHost process.
- Lack of or corrupted NCI configuration files. A corrupted NCI configuration file is not fixable; it must be deleted. An effort was made to minimize this problem starting with NCI version 1.3.x. It is less likely for this to occur with NCI 2.x or later.
- Cryptography module downgrade.

Error -670 Error creating/fetching security domain key

This error is not unique to OpenText eDirectory 8.6.0, but was first reported during OpenText eDirectory version 8.6.0 upgrade testing, probably because servers are not rebooted during the OpenText eDirectory version 8.6.0 upgrade, but directory services is restarted. The problem is duplicated in other environments by restarting directory services (without rebooting and allowing NCI to reinitialize) on servers listed in the W0 object.

Workarounds:

- Avoid restarting directory services on the servers listed in the W0 object without also initializing NCI.

- Restart the server identified by the W0 object before requesting the security domain key (A restart allows NCI to reinitialize, but you still need to be careful not to restart directory services).
- Upgrade to NCI version 2.4 or later.

1.8. Documentation updates

The documentation was updated on the following dates:

- [February 2018](#)
- [January 2016](#)
- [August 2008](#)

1.8.1. February 2018

Updates were made to the following sections. The changes are explained below.

Location	Change
Entire Book	NICI version was updated as part of the NICI 3.1 release.

1.8.2. January 2016

Updates were made to the following sections. The changes are explained below.

Location	Change
Entire Book	Revamped the guide for logical organization of the content.

1.8.3. August 2008

Updates were made to the following sections. The changes are explained below.

Overview

Location	Change
Entire Book	Made editorial changes and updated the guide to current Novell documentation standards.
Installing NCI	Added information to this section, including information regarding the following configuration files: - xmgrcfg.wks - nicifk.new - set_server_mode - set_server_mode.bat



© Copyright 2026 Open Text

For more info, visit <https://docs.microfocus.com>
